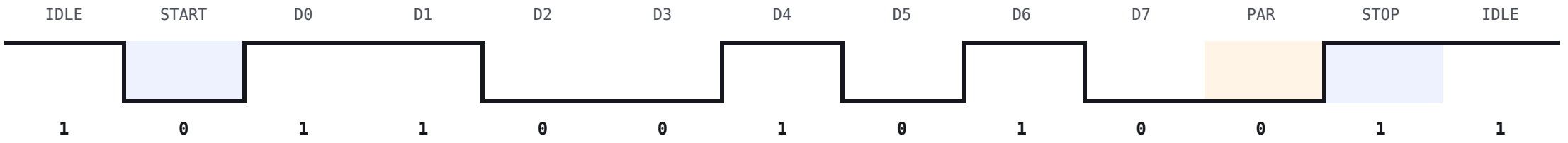
UART Fundamentals & Frame Format

01

Async serial, no shared clock — every frame re-synchronises on its own start edge.

ONE FRAME ON THE WIRE — 8E1, BYTE 0X53, LSB FIRST



0x53 = 0101_0011 → sent D0..D7 = 1,1,0,0,1,0,1,0. Four 1s, so even parity bit = 0. Receiver samples every bit at its middle (16× oversampling: edge + 8 ticks, then every 16 ticks).

CORE CONCEPTS

- **Asynchronous:** no clock wire. TX and RX each run a local baud clock that must agree within a few %.
- **Two signals:** TX → RX crossed between devices, plus common ground. Full-duplex, point-to-point.
- **Idle = 1 (mark).** A falling edge is the only way a frame can begin.
- **Start bit = 0:** gives the receiver a fresh timing reference every frame, so drift never accumulates across frames.
- **Data:** 5–9 bits (8 typical), **LSB first**.
- **Parity (optional):** none / even / odd / mark (1) / space (0).
- **Stop:** 1, 1.5 or 2 bits of 1 — guarantees a high level before the next falling start edge.
- **Notation:** 8N1 = 8 data, no parity, 1 stop. 7E1, 8O2 likewise.

NUMBERS TO KNOW COLD

8.68 µs
bit time @ 115200 baud

104.17 µs
bit time @ 9600 baud

10 bits
per byte in 8N1 → 80% efficiency

11,520 B/s
max payload, 115200 8N1

16×
typical RX oversampling

≈ ±2%
safe per-side clock error (8N1)

Baud = symbols/s. For UART one symbol = one bit, so baud = bit rate — but not throughput.

DEEP DIVE — HOW THE RECEIVER LOCKS ON

1. Synchronise RX to the local clock (2-FF synchroniser — RX is an async input).
2. Detect 1→0 on the synchronised line; start a 0–15 tick counter.
3. At tick 8 (mid start bit) re-sample. Still 0? Real start. Back to 1? **False start** — glitch rejected, return to idle.
4. Sample every 16 ticks: D0..Dn, parity, stop — always mid-bit, maximum margin to both edges.
5. Stop must read 1, else **framing error**. Then hunt for the next falling edge.

Some UARTs take 3 samples around mid-bit (ticks 7/8/9) and majority-vote for noise immunity.

UART VS SPI VS I2C

	UART	SPI	I2C
Clock	none (async)	SCLK from master	SCL, open-drain
Wires	2 (TX, RX)	4 + CS per slave	2 (SDA, SCL)
Topology	point-to-point	1 master, N slaves	multi-master, addressed
Duplex	full	full	half
Error check	parity, framing	none built-in	ACK/NACK

UART is the logic block; RS-232 / RS-485 are electrical layers (RS-232 swings ±3–15 V and is inverted).

TRAPS INTERVIEWERS SET

- "Asynchronous means no clock at all" — wrong. Both sides have clocks; they just aren't shared.
- Bit order: if driver and monitor are both written MSB-first, the test still passes. Check against a known asymmetric byte (0x01, 0x80).
- Throughput ≠ baud: 8N1 spends 2 of every 10 bits on framing.
- Flow control is separate: RTS/CTS (hardware) or XON/XOFF (in-band software).
- Long frames (9 data + parity + 2 stop) shrink the clock-tolerance budget.

WHAT A DV PLAN MUST COVER

- All frame configs: data 5–8, parity ×5 modes, stop 1/1.5/2 — and their cross.
- Every supported baud divisor, plus min and max divisor.
- Back-to-back frames with minimum stop (no idle gap).
- Clock drift injection on the TB side.
- Each error type: parity, framing, break, overrun, false start.
- Loopback mode, FIFO enable/disable, interrupts, reset mid-frame.

INTERVIEW Q&A — SAY THESE OUT LOUD

Q. Why is the idle line high?
So the start bit produces a guaranteed falling edge, and a broken or shorted line (stuck 0) is detectable as a break rather than looking like idle.

Q. Why sample at mid-bit?
It leaves equal margin to both edges, tolerating up to ~half a bit of accumulated timing error by the last bit.

Q. Why 16× oversampling and not 1×?
Without a shared clock the RX must find the edge itself; 16× locates it to within 1/16 bit.

Q. Why doesn't clock drift build up over a long message?
Each frame re-aligns on its own start edge, so error only accumulates across the ~10 bits of one frame.

Q. What is a false start and why reject it?
A low glitch shorter than half a bit. Re-checking at mid-start stops noise from launching a garbage frame.

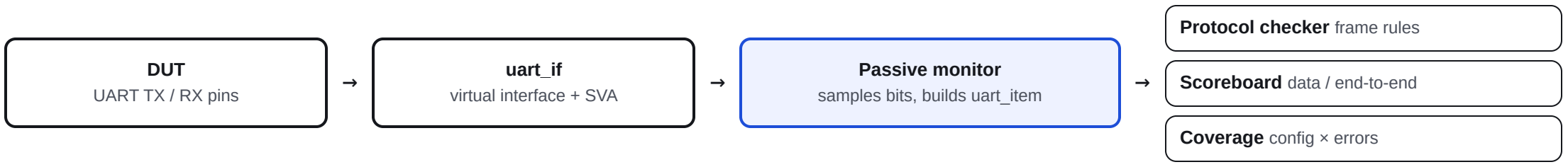
Q. How many errors can parity catch?
Any odd number of flipped bits. Two flips cancel out and go unseen — parity detects, never corrects.

Building the UVM Protocol Checker

02

A passive observer that turns wire activity into transactions — then judges them against the protocol.

WHERE THE CHECKER SITS



Connection: monitor `uvm_analysis_port` → `write()` broadcast to every subscriber (`uvm_subscriber` / `uvm_analysis_imp`). One monitor per direction: TX line and RX line.

BUILDING BLOCKS

- **uart_cfg** (uvm_object): baud, clk freq, data_bits, parity mode, stop_bits, tolerance %, per-check enable knobs. Set once by the test, fetched with `uvm_config_db` in build_phase.
- **uart_item** (uvm_sequence_item): data, parity bit, stop bit(s), start/end timestamps, measured bit times, error flags.
- **Monitor**: watches vif, never drives. Emits one item per frame — frame-level, not bit-level.
- **Checker**: applies rules (baud, framing, parity, break, idle). Reports with distinct message IDs.
- **Agent**: active (driver + sequencer + monitor) for the TB side, passive (monitor only) to observe DUT TX.

UVM PHASES USED

`build_phase`	get cfg + vif from config_db, create ports and sub-components via factory.
`connect_phase`	monitor.ap → checker / scoreboard / coverage exports.
`run_phase`	forever loop: detect start, sample frame, write item.
`check_phase`	leftovers: unmatched items, frame cut by end of test.
`report_phase`	summary: frames seen, errors by type.

MONITOR RUN_PHASE — THE SKELETON EVERY LATER CHECK PLUGS INTO

```
task run_phase(uvm_phase phase);
  forever begin
    uart_item it;
    @(negedge vif.rx);                    // candidate start edge
    t0 = $realtime;
    #(cfg.bit_time/2);                    // move to middle of start bit
    if (vif.rx !== 1'b0) continue;        // false start: glitch, ignore
    it = uart_item::type_id::create("it");
    for (int i = 0; i < cfg.data_bits; i++) begin
      #(cfg.bit_time);  it.data[i] = vif.rx; // LSB first
    end
    if (cfg.parity != PAR_NONE) begin #(cfg.bit_time); it.par  = vif.rx; end
    #(cfg.bit_time);    it.stop = vif.rx;   // checker flags stop == 0
    it.t_start = t0;    ap.write(it);       // broadcast to subscribers
  end
endtask
```

Use `!==` so X/Z on the line is caught. Delay-based sampling is simplest; a clocked 16× sampler mirrors the RTL more closely and catches mid-bit glitches.

DESIGN CHOICES — KNOW THE TRADE-OFF

- **Checks in monitor**: least code, but couples collection with judgement; can't reuse the monitor where errors are expected.
- **Separate checker component**: reusable, checks can be disabled per test, cleaner reporting. Preferred.
- **SVA in the interface**: cycle-accurate, works in formal and emulation, but awkward for multi-bit frame state and config.
- Best practice: **timing checks as SVA**, **content checks in the class**, **data correctness in the scoreboard**.

BUGS TO SPOT IN TUTORIAL-STYLE CODE

- **analysis_port has no get().** It only broadcasts with write(). To pull, use `uvm_tlm_analysis_fifo` and call get() on it.
- **Bit-per-transaction TLM** carries no timing — you cannot check baud from it. Sample the vif.
- **new() on a component inside a module** skips the factory and the hierarchy. Build via `type_id::create` under the env, start with `run_test()`.
- **No objection handling** → sim ends at time 0. The test raises and drops objections, not the monitor.

INTERVIEW Q&A

Q. uvm_object vs uvm_component?
Components are static, live in the hierarchy for the whole sim and have phases. Objects (items, configs, sequences) are transient data with no phases.

Q. analysis_port vs blocking put port?
Analysis is non-blocking, one-to-many, zero-time write(); the monitor never stalls. Put ports are one-to-one and may block.

Q. Checker in RX path only, or both?
Both. TB→DUT checks your driver; DUT→TB checks the design's transmitter. Same monitor class, two instances.

Q. Why should the monitor never drive?
It must observe the same bus in passive agents and at SoC level, where the DUT drives. Any drive breaks reuse.

Q. How do you make the checker configurable per test?
Config object with enable bits and expected params, set from the test through config_db before build.

Q. Where would you use SVA instead?
Idle-high after reset, no X on TX, minimum pulse width — local, timing-shaped rules that also run in formal.

Baud Rate Check

Is every bit the width it should be — and how wrong can it be before data breaks?

03

THE FORMULAS

```
T_bit = 1 / baud
```

```
divisor = round( f_clk / (16 × baud) )
```

```
actual = f_clk / (16 × divisor)
```

```
error % = (actual − target) / target × 100
```

The baud generator is a counter that emits one tick every *divisor* clocks; 16 ticks make one bit. Integer division is why real UARTs are never exactly on target.

WORKED EXAMPLE — 50 MHZ CLOCK, 16×

Target	Exact div	Used	Actual	Error
9600	325.52	326	9585.9	−0.15%
115200	27.13	27	115740.7	+0.47%
921600	3.39	3	1041666.7	**+13.0% ✕**

High baud on a low clock fails — fixes: fractional divisor, 8× oversampling, or a faster / baud-friendly crystal (e.g. 1.8432 MHz divides exactly).

DEEP DIVE — WHERE THE ±2% RULE COMES FROM



Lines = mid-bit sample points, timed from one start edge. Error grows linearly; the stop-bit sample (9.5 T after the edge in 8N1) is the worst case.

1. Last sample must stay inside its bit: drift < 0.5 T over 9.5 T → **< 5.26%**.

2. Edge detection is only good to 1/16 bit: (0.5 − 0.0625) / 9.5 → **≈ 4.6%** total.

3. Split between TX and RX clocks → **≈ ±2.3% each**; rise times and noise eat more → design to **±2%**.

4. Longer frames (8 data + parity + 2 stop) put the last sample later → tighter budget.

ROBUST CHECKER — MEASURE EVERY EDGE GAP, NOT A COUNT

```
forever begin
  @(vif.rx);  t = $realtime;                      // any edge
  if (in_frame) begin
    gap = t - t_prev;
    n   = $rtoi(gap / cfg.bit_time + 0.5);        // nearest whole number of bits
    err = gap - n * cfg.bit_time;
    if (err < 0) err = -err;
    if (n == 0)                                    // pulse shorter than half a bit
      `uvm_error("UART_GLITCH", $sformatf("%0.1f ns pulse", gap))
    else if (err > n * cfg.bit_time * cfg.tol)      // tol e.g. 0.02
      `uvm_error("UART_BAUD", $sformatf("gap %0.1f ns ~ %0d bits, off %0.2f%%",
                 gap, n, 100.0 * err / (n * cfg.bit_time)))
  end
  t_prev = t;
end
```

Every bit boundary that has an edge gets checked, and long runs of equal bits are checked as n × T. Use `realtime` and 1 ps precision: 1e9 / 115200 = 8680.555… ns would round badly at 1 ns.

WEAK APPROACHES TO CALL OUT

- **"Count transitions in a time window"** — transition count depends on the data. 0x00 and 0xFF have almost none, so the measured rate is meaningless.
- **"Count loop iterations and compare to the expected count"** — the loop always runs the expected number of times. It checks the checker, never the DUT.
- **Sampling only mid-bit** hides small drift: the value is still right. You need edge timestamps to see the error before it breaks data.
- **Hard-coded baud parameter** — take it from the config object so one checker covers every divisor.

TESTS & COVERAGE

- **DUT TX:** program every divisor (incl. 1 and max); checker confirms measured T within tolerance.
- **DUT RX:** TB driver gets a drift knob. Sweep 0, ±1, ±2, ±3, ±4.5, ±6%: data correct inside the budget, framing errors expected beyond it.
- **Change baud mid-traffic** (divisor latch write) — first frame after must use new rate.
- **Coverage:** baud bins × frame format; drift bins × pass/fail; divisor min / max / typical.
- **Autobaud** (if supported): measure the start bit of a known char such as 0x55 (alternating bits).

INTERVIEW Q&A

Q. Mismatched baud — what does the RX see?
Small mismatch: nothing. Larger: late bits sampled in the wrong cell → wrong data, then stop bit read as 0 → framing error.

Q. How would you inject drift in the TB?
Driver uses bit_time × (1 + drift) from a randomised config field, or drives from a separate clock with ppm offset.

Q. Why measure edges instead of trusting data?
Data passes until the margin is gone. Edge timing shows a 3% error long before it becomes a data failure.

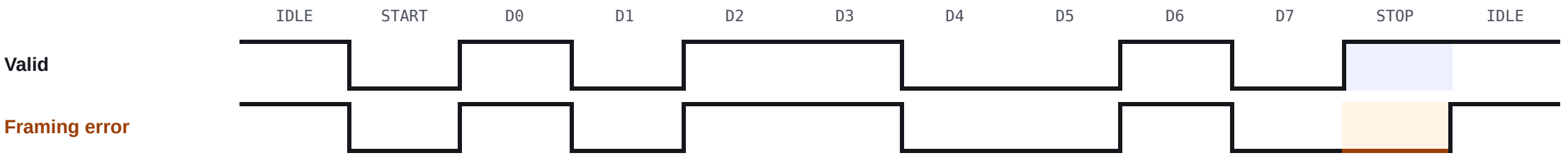
Q. Why 16× and not 8×?
Finer edge location (1/16 vs 1/8 bit) leaves more drift budget. 8× allows higher baud from the same clock.

Framing Error Check

04

The stop bit is the frame's closing bracket. If it reads 0, the receiver lost its place.

GOOD FRAME VS FRAMING ERROR (8N1)



Precise definition: the receiver samples the stop bit at its middle and reads 0 instead of 1. It is a statement about the *stop bit*, not about matching "codes" at both ends — a common fuzzy explanation interviewers will push on.

ROOT CAUSES

- **Baud mismatch** — sample points slide until the stop sample lands in the next start bit.
- **Format mismatch** — TX 7E1 vs RX 8N1: RX reads one bit too many or too few, so "stop" is really data.
- **Noise** on the stop bit.
- **Break** — line held low for a whole frame or more (part 06).
- **TX reset / disable mid-frame**, or a premature next start (TX sends less than 1 stop bit).
- **Lost sync** — RX locked onto a 1→0 inside data after an earlier error.

WHAT THE DUT SHOULD DO (16550-STYLE)

- Set **LSR.FE** (bit 3) for that character; it's tied to the char at the FIFO head, revealed when that char is read.
- Still store the character (marked as errored); **LSR bit 7** = "error somewhere in RX FIFO".
- Raise the **receiver line status interrupt** (highest priority) if enabled.
- **Resync policy is spec-specific:** some UARTs treat the low "stop" as the next start bit, others wait for the line to go high first. The checker must mirror whichever the spec says.
- With 2 stop bits, many receivers check only the **first**; TX must still send both.

CHECKER + ERROR-INJECTION HOOKS

```
// monitor: after data (+ parity)
for (int s = 0; s < cfg.stop_bits; s++) begin
  #(cfg.bit_time);
  it.stop[s] = vif.rx;
end
it.fe = (it.stop[0] !== 1'b1);              // RX rule: first stop bit decides

// checker: write() from monitor
if (it.fe) begin
  if (it.fe_expected) `uvm_info ("UART_FE_EXP", "injected framing error seen", UVM_MEDIUM)
  else                `uvm_error("UART_FE",     $sformatf("stop=0 after data 0x%0h", it.data))
end
if (cfg.check_all_stops && it.stop[1] === 1'b0)  // TX-side rule for 2 stop bits
  `uvm_error("UART_FE2", "second stop bit low")

// driver knob: corrupt the stop bit on purpose
vif.rx <= (tr.inject_fe) ? 1'b0 : 1'b1;  #(cfg.bit_time);
```

Carry intent in the item (`inject_fe → fe_expected`) so negative tests pass cleanly and unexpected FEs still fail.

TEST PLAN & COVERAGE

1. Directed: single FE frame, then read LSR via RAL — FE set, data present, clears on read.
2. Random mix of valid / errored frames at a set injection rate; scoreboard tracks expected flags per char.
3. **Recovery:** FE frame immediately followed by a valid frame (no idle) — must decode correctly.
4. FE with each format: data 5–8 × parity × stop 1/2.
5. Format mismatch test: TB 7E1 into DUT 8N1.
6. Coverage cross: fe × format × back-to-back × FIFO on/off × interrupt enabled.

EDGE CASES THAT FIND BUGS

- Framing error vs **false start**: a glitch rejected at mid-start is not an FE — no flag, no char.
- 0x00 with a low stop bit is exactly a frame of zeros → candidate **break**; duration decides.
- FE on the char that fills the FIFO — does the error flag land on the right entry?
- Cascades: one FE can mis-sync the next frames. Count root causes, not every downstream error.
- Reset asserted mid-frame: no FE should be reported for the cut-off frame after reset.

INTERVIEW Q&A

Q. Framing error vs parity error?
Framing = structure broken (stop bit). Parity = structure fine, content suspect. They can occur together.

Q. Why do baud mismatches show up as framing errors first?
The stop bit is sampled last, so it has the most accumulated drift.

Q. How do you keep negative tests from failing?
Mark the item as expected-error, or demote the specific ID with a report catcher for that test only — never globally.

Q. What if the scoreboard expects data but FE was flagged?
Follow the spec: usually data is stored with an error tag. The scoreboard compares data and the error flag together.

Parity Error Check

05

One extra bit that makes the count of 1s even (or odd). Cheap, single-error detection — nothing more.

FIVE PARITY MODES

Mode	Parity bit P	Check passes if
None	no bit sent	—
Even	`P = ^data`	`^{data,P} == 0`
Odd	`P = ~^data`	`^{data,P} == 1`
Mark	always 1	`P == 1`
Space	always 0	`P == 0`

16550 LCR: bit 3 PEN (enable), bit 4 EPS (even select), bit 5 stick parity (mark/space). Error shows in LSR.PE, bit 2.

WORKED EXAMPLES

Byte	Binary	1s	Even	Odd
0x53	0101_0011	4	0	1
0x57	0101_0111	5	1	0
0x00	0000_0000	0	0	1
0xFF	1111_1111	8	0	1

Sanity check: odd parity is always the inverse of even parity for the same data.

DEEP DIVE — WHAT PARITY CAN AND CANNOT SEE

1 bit flipped
Detected — count changes parity.

2 bits flipped
Missed — flips cancel. Only the scoreboard catches it.

P itself flipped
Detected — looks the same as a data flip.

Which bit?
Unknown. Parity detects, never locates or corrects.

Formally: parity gives a minimum Hamming distance of 2 — detects any odd number of errors, corrects none. Need correction → Hamming / ECC. Need burst detection → CRC at a higher layer.

CHECKER — COMPUTE OVER THE CONFIGURED WIDTH ONLY

```
function bit exp_parity(bit [8:0] data);
  bit x = 1'b0;
  for (int i = 0; i < cfg.data_bits; i++) x ^= data[i];   // 5..8 bits, never blindly 8
  case (cfg.parity)
    PAR_EVEN : return x;
    PAR_ODD  : return ~x;
    PAR_MARK : return 1'b1;
    PAR_SPACE: return 1'b0;
  endcase
endfunction

// in write(uart_item it)
if (cfg.parity != PAR_NONE) begin
  it.pe = (it.par !== exp_parity(it.data));
  if (it.pe && !it.pe_expected)
    `uvm_error("UART_PE", $sformatf("data 0x%0h par %b exp %b", it.data, it.par, exp_parity(it.data)))
end
```

Two ways to check — pick one, don't mix: compare P to the computed value, *or* XOR data with P and compare the result to 0 (even) / 1 (odd). XOR-ing P in and then comparing to P is a classic bug.

TESTS & COVERAGE

- Injection enum in the item: `{NONE, FLIP_PAR, FLIP_1_DATA, FLIP_2_DATA}`.
- Expect: PE set for FLIP_PAR and FLIP_1_DATA; PE clear but data mismatch for FLIP_2_DATA.
- Each mode with data_bits 5, 6, 7, 8 — the width bug only shows below 8.
- Change parity mode between frames (LCR write while idle).
- Cross: parity_mode × data_bits × injection × FIFO position.
- DUT TX side: verify generated P for all modes, incl. all-0 and all-1 data.

COMMON BUGS (DUT AND TB)

- Parity computed over all 8 bits when 7 are configured (stale MSB leaks in).
- Even / odd swapped — invisible if TB model has the same swap. Check against a hand-computed table.
- Mark / space ignored: hardware sends computed parity instead of a constant.
- Parity bit position off by one → RX also reports framing errors.
- PE flag attached to the wrong FIFO entry.

INTERVIEW Q&A

Q. Why do mark and space parity exist?
Used as a 9th bit: in multi-drop links the parity slot flags "address byte" vs "data byte".

Q. If parity misses double errors, why verify them?
To prove the DUT does *not* flag them and that your scoreboard, not the checker, catches the corruption.

Q. How is parity computed in hardware?
An XOR tree (SV reduction ^), or a 1-bit toggle flop updated as each bit shifts out.

Q. Can a frame have both PE and FE?
Yes. Both flags are set for that character; report both, but treat the FE as the likely root cause.

Break Condition

Line held low for longer than a whole character — a deliberate signal, not just a bad frame.

06

WHAT IT LOOKS LIKE (8N1 — ONE CHARACTER TIME = 10 T)



Definition: RX input stays at 0 for longer than one full character (start + data + parity + stop). Every sampled bit — data, parity, stop — reads 0, and the line keeps going.

EXPECTED DUT BEHAVIOUR (16550-STYLE)

- **RX:** set **LSR.BI** (bit 4); FE is typically set with it (spec-dependent).
- Load **one** 0x00 character into the FIFO per break — not one per elapsed frame time.
- Then wait for the line to return to **mark (1)** and a fresh valid start before receiving again.
- Raise the receiver line status interrupt if enabled.
- **TX:** LCR bit 6 "set break" forces TX low until cleared. Safe sequence: wait for TX empty, set break, hold the required time, clear.

WHY BREAK EXISTS

- **Attention / out-of-band signal** — e.g. console "break" to drop into a debugger or bootloader.
- **LIN bus** — each frame begins with a sync break of at least 13 dominant (low) bits.
- **Autobaud / resync** — a guaranteed reset point that can't occur in normal data.
- **Fault detection** — a cut or shorted line reads as permanent 0, i.e. continuous break.

Break vs framing error: both have stop = 0. A framing error can have any data and the line rises soon after. A break has all-zero samples *and* stays low past the character time. Duration decides.

BREAK DETECTOR — A PARALLEL THREAD WITH A TIMEOUT RACE

```
forever begin
  @(negedge vif.rx);  t0 = $realtime;
  fork begin                                  // isolate: disable fork kills only these
    fork
      wait (vif.rx === 1'b1);                 // line came back: ordinary frame
      #(cfg.char_bits * cfg.bit_time);         // char_bits = 1 + data + par + stop
    join_any
    disable fork;
  end join
  if (vif.rx === 1'b0) begin                  // still low after a full character
    in_break = 1;                             // frame sampler skips FE/baud checks
    wait (vif.rx === 1'b1);
    brk_bits = ($realtime - t0) / cfg.bit_time;
    `uvm_info("UART_BRK", $sformatf("break %0.1f bit times", brk_bits), UVM_MEDIUM)
    if (!brk_expected) `uvm_error("UART_BRK_UNEXP", "unexpected break")
    in_break = 0;
  end
end
```

The outer `fork begin … end join` wrapper matters: a bare `disable fork` also kills every other thread this process spawned — a favourite SV interview question.

TEST PLAN — DURATIONS AROUND THE THRESHOLD

0.9 char	Not a break. Data 0x00 + framing error expected.
1.0 char	Boundary — spec decides. Classic corner bin.
1.5 / 2 char	Break: BI set, one 0x00 in FIFO.
20+ char	Still exactly one char and one BI — no repeats.
break → char	Valid frame right after mark returns decodes cleanly.
TX set-break	Set mid-character and when idle; check TX low time and clean recovery.

BUGS THIS CATCHES

- RX pushes a 0x00 into the FIFO every character time during a long break → FIFO floods, overrun.
- RX restarts on the first sample after break without waiting for mark → junk frame.
- BI asserted but FE not (or vice versa) against the spec.
- Checker floods the log with FE + baud errors during an intended break — suppress while `in_break`.
- Break while FIFO is full: is the break char dropped and OE set?

INTERVIEW Q&A

Q. How is break different from sending 0x00?
0x00 has a stop bit of 1 after 9 low bits. Break has no stop bit — the line stays low past the whole character.

Q. Why only one character per break?
Break is one event. Repeated 0x00 chars would look like data and fill the FIFO.

Q. How do you detect break in the TB?
Race a "line went high" wait against a one-character timeout; whichever finishes first decides.

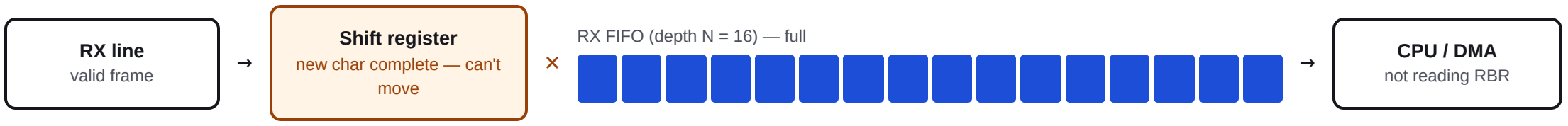
Q. Where else have you seen break used?
LIN's 13-bit sync break, bootloader entry, debug consoles.

Overrun Error

The wire was perfect — the software was too slow. A character arrives with nowhere to go.

07

THE DATA PATH — AND WHERE THE CHARACTER IS LOST



Definition: a new character completes in the receive shift register while the holding register / FIFO is still full. One character is lost and **LSR.OE** (bit 1) is set. Nothing is wrong on the wire — so a line monitor alone can never detect it.

WHICH CHARACTER IS LOST? (16550-STYLE)

FIFO off	New char overwrites the unread one in RBR. Old data lost.
FIFO on	Char in the shift register is overwritten and never enters the FIFO. FIFO contents stay valid.
OE timing	Flag rises as soon as the overrun happens; cleared by reading LSR.

Exactly which char is dropped, and at which count (N+1 or N+2) OE rises, is spec-specific — make the model take it from the spec, and cover the boundary.

HOW SYSTEMS PREVENT IT

- **RX FIFO trigger level** (1 / 4 / 8 / 14): interrupt early enough for software to drain.
- **Character timeout** interrupt so a few leftover bytes don't sit unread.
- **Hardware flow control:** auto-RTS deasserts near full so the sender pauses (CTS).
- **DMA** for sustained high baud rates.
- Verify each mechanism: with auto-RTS on, a burst must *never* overrun.

PREDICT IT IN THE SCOREBOARD — TWO INPUTS, ONE MODEL

```
uart_item  rx_q[$];      // model of the DUT RX FIFO
bit        exp_oe;

function void write_line(uart_item it);      // from line monitor (frame complete)
  if (rx_q.size() < cfg.fifo_depth) rx_q.push_back(it);
  else begin exp_oe = 1; n_lost++; end      // FIFO-mode rule: newest char dropped
endfunction

function void write_bus(bus_item b);        // from APB/AXI monitor or RAL predictor
  if (b.is_read && b.addr == RBR) begin
    uart_item exp = rx_q.pop_front();
    if (b.data[7:0] !== exp.data) `uvm_error("UART_SB", "RBR data mismatch")
  end
  if (b.is_read && b.addr == LSR) begin
    if (b.data[1] !== exp_oe) `uvm_error("UART_OE", $sformatf("OE=%b exp %b", b.data[1], exp_oe))
    exp_oe = 0;                              // OE clears on LSR read
  end
endfunction
```

Overrun lives inside the DUT, so it needs **both** sides: the serial monitor (chars in) and the register-bus monitor (chars out). Use `uvm_analysis_imp_decl` to give the scoreboard two write methods.

TEST PLAN & COVERAGE

1. No reads; burst N, N+1, N+2, N+10 chars. OE exactly at the spec'd count, FIFO keeps the first N.
2. FIFO disabled (depth 1): two chars without a read.
3. Reader slower than arrival rate — steady-state overruns.
4. **Race:** RBR read lands in the same cycle the new char completes.
5. Read LSR → OE clears; second overrun sets it again.
6. Auto-RTS on: same burst, zero overruns, TX pauses.
7. Cover: FIFO level at arrival {0, mid, N−1, N} × FIFO on/off × flow control × error-tagged char.

MODELLING PITFALLS

- **Same-cycle race:** a transaction-level model can't know if the read or the push won. Either model at cycle level, or accept both outcomes within a small window and cover which one happened.
- **"Char complete" point:** RTL may push at mid-stop-bit, your monitor at end of stop. Align the model with the RTL's push point.
- Wrong lost-char assumption (oldest vs newest) → every later compare mis-aligns.
- Forgetting that an LSR read clears OE *and* other sticky bits.

INTERVIEW Q&A

Q. Why can't a protocol checker on the line catch overrun?
The serial stream is perfectly valid. The loss happens between the FIFO and the CPU — you need the bus side too.

Q. How do you force overrun in a test?
Mask RX interrupts, stop the reader sequence, and drive more than FIFO depth back-to-back frames.

Q. Overrun vs underrun?
Overrun: RX data arrives faster than it's read. TX "underrun" just means TX empty — the line idles, THRE interrupt fires, not an error.

Q. How would you verify the read/arrive race?
Constrain read timing relative to the stop bit, sweep the offset by a cycle at a time, and cover the exact-coincidence bin.

Data Validity & Idle Line

Right bits, right order, right width — and a line that stays quietly high when nothing is sent.

08

DATA VALIDITY — WHAT TO CHECK

- **Bit order:** LSB first on the wire. D0 follows the start bit.
- **Width:** exactly 5, 6, 7 or 8 data bits as configured — no extra, no missing bit.
- **Unused bits:** with 5–7 bit words, upper RBR bits read 0 (check the spec).
- **No X / Z** sampled in data — always an error, never "don't care".
- **End-to-end:** byte written to THR on one side = byte read from RBR on the other, in order.

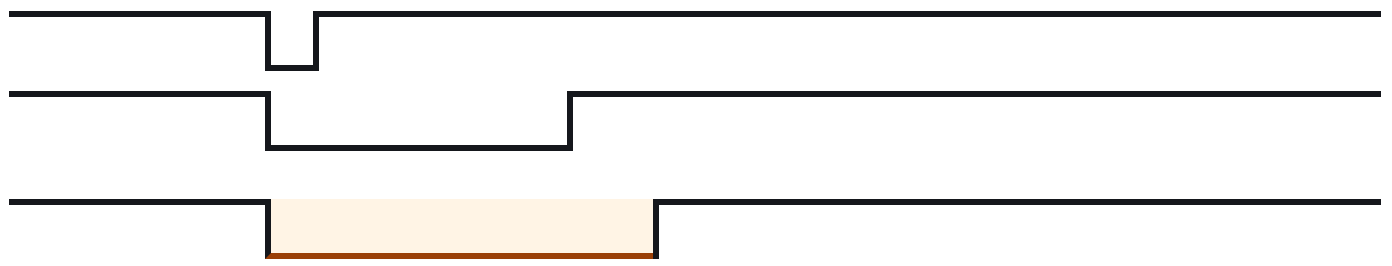
THE SYMMETRIC-BUG TRAP

If the TB driver *and* monitor both send MSB-first, a DUT that also does MSB-first passes every test. Loopback tests have the same blind spot: DUT TX → DUT RX hides any mistake made in both.

Break the symmetry:

- One directed test checks the **raw waveform** of 0x01: line must go high in the bit right after start.
- Asymmetric patterns: 0x01, 0x80, 0x0F, 0xF0, 0xA5, walking 1s and 0s.
- Build driver and monitor from the spec independently, ideally by different people.

IDLE LINE — GLITCHES NEAR THE START SAMPLE POINT



Glitch 1/16 bit	Rejected — high at mid-start.
Glitch 7/16 bit	Rejected — false start.
Glitch 9/16 bit	Accepted as start → all-1 data, so the frame decodes 0xFF (stop is high, no FE). Noise becomes data.

The exact accept/reject boundary depends on the sampler (single sample at tick 8, or 3-sample majority) — that boundary is a coverage bin, and the checker should flag any sub-half-bit pulse on a TX line.

IDLE-LINE RULES

- **TX = 1 in and right after reset.** A low blip on reset looks like a start bit to the partner.
- **Never X / Z** on TX once out of reset.
- **Gap between frames ≥ configured stop bits** — the next start may not come early.
- **Reset or disable mid-frame:** TX returns to 1 at once; RX throws away the partial frame silently.
- **Character timeout** (16550: about 4 character times of silence with data in FIFO) — RX tells software leftover bytes are waiting.

SVA IN THE INTERFACE

```
// TX held at mark while in reset
a_idle_rst: assert property (
  @(posedge clk) !rst_n |-> tx === 1'b1);

// no unknowns once running
a_no_x: assert property (
  @(posedge clk) disable iff (!rst_n)
  !$isunknown(tx));

// a low pulse on TX lasts at least one bit
a_min_low: assert property (
  @(posedge clk) disable iff (!rst_n)
  $fell(tx) |-> (!tx)[*(BIT_CLKS-1)]);
```

`BIT_CLKS` = clocks per bit from the divisor. Use a sampled strobe instead of `clk` if the divisor changes at runtime.

COVERAGE MODEL

Data values
0x00, 0xFF, 0x55, 0xAA, walking-1, walking-0, random — crossed with width 5–8.

Inter-frame gap
back-to-back (0 extra), 1 bit, short, long idle, > char-timeout.

Line disturbances
glitch width bins around half-bit, reset mid-frame, enable/disable mid-frame.

INTERVIEW Q&A

Q. All tests pass but the chip talks garbage to a real PC. Why?
Likely a symmetric bug — bit order, parity sense or polarity wrong in both DUT and TB. Check one frame against the spec by hand.

Q. What does a character timeout solve?
With trigger level 8, the last 3 bytes of a message never reach the trigger. Timeout tells software to collect them.

Q. Why must TX be high during reset?
The partner is already running; any low pulse is a start bit and produces a junk byte or framing error on its side.

Q. Why treat X on the line as an error rather than ignore it?
X usually means uninitialised or multiply-driven logic — a real silicon risk hidden by simulation optimism.

Error Reporting & Full Integration

Every check in one place — what it sees, who catches it, and how a failure gets reported cleanly.

09

ERROR TAXONOMY — THE ONE TABLE TO MEMORISE

Error	On wire?	Caught by	DUT flag (LSR)	Inject with
Baud / timing	yes	checker: edge-gap timing	none (shows as FE)	driver drift %
Framing	yes	checker: stop sample	FE · bit 3	force stop = 0
Parity	yes	checker: XOR over width	PE · bit 2	flip P or 1 data bit
Break	yes	checker: low-time timer	BI · bit 4	hold low > 1 char
Overrun	**no**	scoreboard: line in + bus out	OE · bit 1	stop reading, burst
False start	yes	checker: pulse width	none (silently dropped)	short low glitch
Data / order	partly	scoreboard + directed waveform	—	asymmetric bytes
Idle / X	yes	SVA in interface	—	reset mid-frame

Other LSR bits: DR (0) data ready · THRE (5) TX holding empty · TEMT (6) TX fully empty · bit 7 error somewhere in RX FIFO. Error bits are sticky and clear on LSR read.

THE COMPLETE ENVIRONMENT

`uart_test → sets uart_cfg (format, baud, tolerance, check enables, error knobs) · raises objections · adds report catchers`

`uart_env`

- **Serial agent (active)** — sequencer · driver with error knobs · monitor — drives DUT RX
- **Serial agent (passive)** — monitor only — watches DUT TX
- **Bus agent + RAL** — APB/AXI-lite to THR, RBR, LCR, LSR, IER, FCR
- **Virtual sequencer** — coordinates register config with serial traffic
- **Protocol checker** — baud · framing · parity · break · glitch
- **Scoreboard** — FIFO model · data in order · OE · error-flag per char
- **Coverage collector** — format × baud × error × FIFO level
- **uart_if + SVA** — idle-high, no-X, minimum pulse width

REPORTING DONE RIGHT

- **One ID per check** (UART_FE, UART_PE, UART_BRK, UART_OE, UART_BAUD) so logs can be filtered and severities overridden per ID.
- **Severity:** DUT violation = `uvm_error`; TB-injected and seen = info; avoid warnings nobody reads.
- **Message content:** time, direction (TX/RX), data, which bit, measured vs expected.
- **Suppress cascades:** after FE or break, mark "resyncing" and hold follow-up checks until the line is idle.
- **report_phase summary:** frames seen, per-error counts, expected vs unexpected.
- **Knobs:** every check has an enable bit in cfg — but never disable one to make a test pass.

EXPECTED ERRORS — A REPORT CATCHER

```
class fe_expect_catcher extends uvm_report_catcher;
  `uvm_object_utils(fe_expect_catcher)
  int seen;
  function new(string name = "fe_expect_catcher");
    super.new(name);
  endfunction
  function action_e catch();
    if (get_id() == "UART_FE" &&
        get_severity() == UVM_ERROR) begin
      set_severity(UVM_INFO);  seen++;
    end
    return THROW;
  endfunction
endclass
// test: uvm_report_cb::add(null, catcher);
// check_phase: seen == number injected
```

Count what you demote and compare to what you injected — otherwise a missing error passes silently.

RAPID-FIRE REVISION — THE WHOLE SERIES IN 10 LINES

1. Idle 1, start 0, 5–8 data LSB-first, optional parity, 1–2 stop of 1.

2. RX: 16× oversample, confirm start at mid-bit, sample every 16 ticks.

3. Tolerance: last sample at 9.5 T → ~4.6% total → design to ±2% per side.

4. Check baud by timing every edge gap as n × T, never by counting transitions.

5. Framing = stop bit sampled 0. Resync policy comes from the spec.

6. Parity catches odd-count flips only; compute over configured width.

7. Break = low longer than one char; one 0x00 + BI per break, then wait for mark.

8. Overrun is invisible on the wire — model FIFO with line + bus monitors.

9. Symmetric bugs pass loopback; verify one raw waveform against the spec.

10. Unique IDs, expected-error flags or catchers, cascade suppression, summary in report_phase.

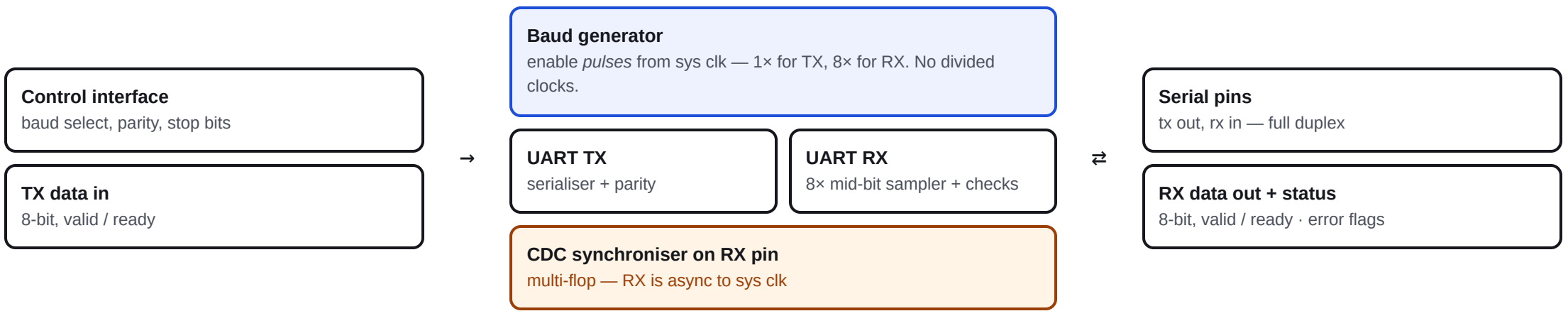
UART · DV INTERVIEW CARD · BONUS · REAL IP

Chipmunk Logic UART Controller

Mapping cards 01–09 onto a real open-source IP — what changes, what to verify, what to ask.

IP

ARCHITECTURE — ONE CLOCK, ONE RESET, NO FIFO



PUBLISHED FEATURE SET (V1.1)

Data width	8 bits only, LSB first
Parity	none / odd / even — no mark or space
Stop bits	1 or 2
Oversampling	**8×** (not 16×), sample at mid-bit
Baud	configurable; typical 1200–115200
Clock tolerance	design target ±5%
Buffering	none inside — valid / ready for an external FIFO
Errors	parity and framing flags; break and overrun not described
Board test	Basys-3 ↔ Arduino loopback, 115200, odd parity, 1 stop

v1.2 (Feb 2024) adds unlisted "upgraded features"; its file list swaps reset_sync for areset_sync and grows baud_gen and top.

DEEP DIVE — IS ±5% REALLY SAFE AT 8×?

The ±5.3% figure is 0.5 bit ÷ 9.5 bits — the limit with *perfect* edge detection. Real limits are tighter:

`8× edge uncertainty`	`1/8 = 0.125 bit`
`8N1, last sample 9.5 T`	`(0.5−0.125)/9.5 ≈ 3.9%`
`8O1, last sample 10.5 T`	`0.375/10.5 ≈ 3.6%`
`16× for comparison (8N1)`	`0.4375/9.5 ≈ 4.6%`

Those are **combined** TX + RX budgets. So "±5%" only holds if the far end is near-perfect. A drift sweep (card 03) with parity on is the test that settles it — a strong talking point.

HOW EACH CARD CHANGES FOR THIS IP

01 Fundamentals	Same frame; RX locks with 8 ticks/bit — confirm start at tick 4, sample every 8.
02 Checker	No register bus: pin monitors + **valid/ready stream monitors** on both data ports feed the scoreboard.
03 Baud	divisor = f_clk ÷ (8 × baud) for RX pulses; check TX pulse spacing and the pulse counter width vs lowest baud.
04 Framing	Check the status flag's timing: per-byte with data valid, or sticky until cleared?
05 Parity	Only none / odd / even — drop mark/space bins; data width fixed at 8, so no width bug.
06 Break	Not a feature: expect 0x00 + framing error. Check RX doesn't emit a byte every frame time while held low.
07 Overrun	No FIFO → becomes **backpressure**: a new byte completes while out-ready is low. Drop, overwrite or flag? Read the RTL.
08 Idle / CDC	Synchroniser flops must **reset to 1**; reset-to-0 makes a fake falling edge (false start) at reset release.
09 Reporting	Status flags instead of LSR: map each checker ID to a flag; cover flag × parity mode × stop bits.

VALID / READY RULES — SVA

```
// once raised, valid holds and data is stable
a_hold: assert property (@(posedge clk)
  disable iff (!rst_n)
  (tx_valid && !tx_ready) |=>
    (tx_valid && $stable(tx_data)));

// a handshake starts exactly one frame
a_start: assert property (@(posedge clk)
  disable iff (!rst_n)
  (tx_valid && tx_ready) |-> ##[1:$] $fell(tx));

// TX line idles high out of reset
a_idle: assert property (@(posedge clk)
  !rst_n |-> tx === 1'b1);
```

Also cover: back-to-back handshakes, ready low for many frames, valid toggling with no handshake.

OPEN QUESTIONS TO SETTLE FROM THE RTL

1. RX output ready low when a byte completes — drop, overwrite or flag?
2. Framing / parity flags: pulse with data valid, or sticky?
3. Number of sync stages on RX, and their reset value.
4. Is the start re-checked at mid-bit (false-start rejection)?
5. With 2 stop bits, does RX check both?
6. Can config change mid-frame — and what happens?
7. Baud counter width → lowest reachable baud at a given f_clk.
8. What does the synthesisable TB check, and what does it skip?

INTERVIEW Q&A ON THIS DESIGN

Q. Why baud enable pulses rather than a divided clock?
Everything stays in one clock domain: no generated clocks, no extra skew or CDC, simple STA and FPGA-friendly.

Q. 8× vs 16× oversampling?
8× halves the sample-pulse rate (higher max baud, less toggling) but doubles edge uncertainty, shrinking drift tolerance.

Q. Why does RX need a synchroniser but TX doesn't?
RX comes from a foreign clock and can change near our clock edge → metastability. TX is driven from our own flops.

Q. No FIFO — how do you test throughput?
Randomise ready on both ports, run continuous traffic, and check no byte is lost or duplicated under backpressure.

Basis: chipmunklogic.com "Design of UART Controller" page · RTL not read (archives encrypted) · tolerance figures computed

Pairs with cards 01–09